

GuitarNeXt - Music Genre Classification

Author: Shree Ramachandran Muthukumaran

Instructor: Gady Agam

Course: CS-584-02

11/22/2025

Abstract

This project implements the GuitarNeXt architecture for automated music genre classification using the GTZAN dataset. Audio tracks are converted into mel-spectrograms and input to a lightweight convolutional network inspired by ConvNeXt but optimized for spectrogram analysis. The model was trained for 25 epochs from scratch using a custom Python implementation. Results include a training accuracy of ~74% and validation accuracy of ~67%, supported by precision, recall, F1-score, and confusion matrix analysis. Differences from the original paper are explained in terms of dataset size, preprocessing methodology, and lack of 3-stage pretraining. The project demonstrates the effectiveness and limitations of training CNN architectures on small audio datasets.

1. Introduction

1.1 Problem Statement

Music genre classification is the task of automatically predicting the musical genre of an audio recording. It is widely used in music organization and recommendation systems. Since genres are abstract and loosely defined, genre classification is very difficult and requires extraction of audio details that capture rhythm, timbre, harmony, and other special characteristics. Furthermore, many genres overlap significantly (e.g., rock and metal, hip-hop and pop), making classification very challenging for a computer.

1.2 Motivation

Deep learning models have shown strong performance on spectrogram-based audio classification. However, large models are often too computationally complex for small datasets, which makes them prone to overfitting and difficult to train efficiently. GuitarNeXt has a lightweight architecture designed for spectrogram inputs, providing efficient feature extraction while maintaining strong classification capability. This project aims to implement and evaluate GuitarNeXt on the GTZAN dataset to determine how well a compact model performs when trained from scratch using limited data.

2. Background and Related Work

2.1 Spectrogram-Based Audio Classification

Spectrograms are widely used in audio analysis since they convert a one-dimensional waveform into a two-dimensional representation that captures how energy in different frequency bands evolves over

time. This allows machine learning models to identify musical characteristics to help distinguish between genres. There are a variety of Spectrograms, including Short-Time Fourier Transform spectrograms, chromagrams, and mel-spectrograms. This project uses 128-mel spectrograms as the mel scale provides finer resolution at lower frequencies where most musical information is concentrated. As a result, mel-spectrograms serve as an effective input for deep learning-based genre classification.

2.2 Convolutional Neural Networks for Spectrograms

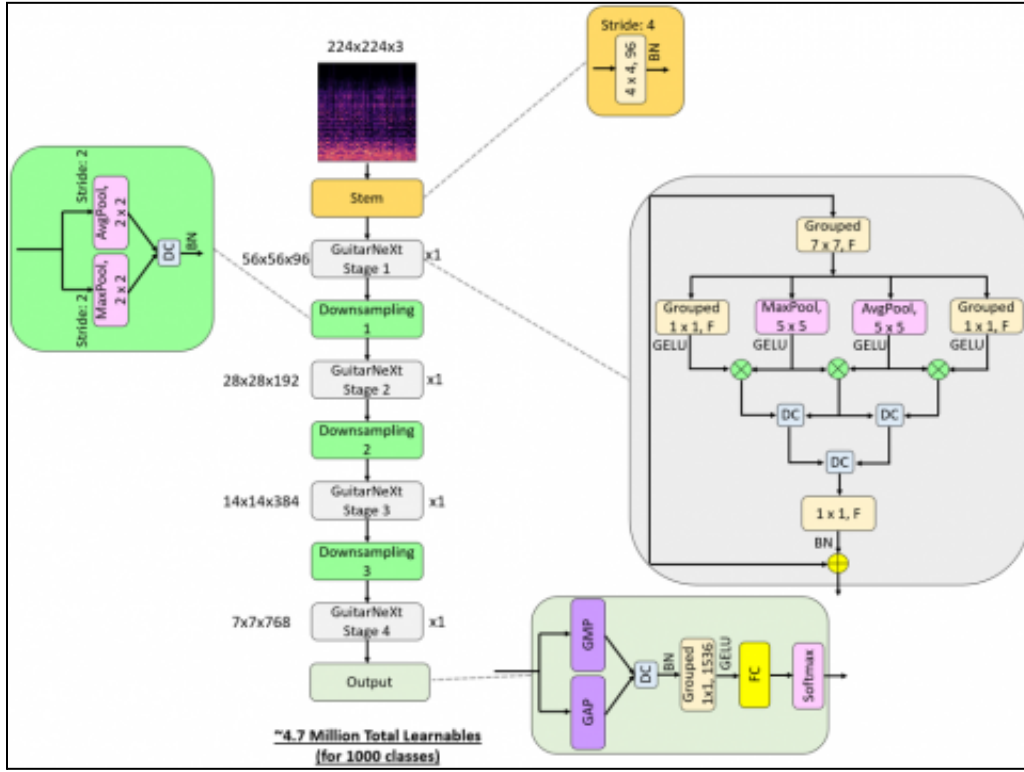
Convolutional neural networks (CNNs) have become a dominant approach for music genre classification due to their ability to learn local and general patterns from images. When applied to spectrograms, CNNs can automatically identify important musical features. Standard CNN architectures such as VGG, ResNet, and ConvNeXt have demonstrated strong performance in various audio classification tasks. GuitarNeXt builds on the principles of ConvNeXt by modifying the architecture to better capture diverse frequency-time relationships found in music.

3. Proposed Method

3.1 Pipeline Overview

The proposed system performs music genre classification from raw audio by converting waveform data into mel-spectrogram images and then using the GuitarNeXt neural network for prediction. Each audio file is first loaded and normalized before being converted into a 128-mel spectrogram. The spectrogram is resized to 224×224 and converted to a three-channel RGB to match the expected input format. These images are then fed into the GuitarNeXt architecture, which extracts time-frequency features using a multi-branch convolutional structure. The model is trained using stochastic gradient descent with a learning-rate scheduling strategy and mild data augmentation. Training runs for up to twenty epochs, and model checkpoints are saved based on validation accuracy. After training, performance is evaluated using multiple metrics, including classification accuracy, precision, recall, F1-score, and confusion matrix analysis, along with visual inspection of the training curves.

3.2 Network Architecture Description



The graphical outline of the recommended GuitarNeXt

The GuitarNeXt architecture consists of four main phases:

1. Stem phase
2. GuitarNeXt phase
3. Downsampling phase
4. Output phase

The stem phase begins with a patchify convolution using a 4×4 kernel with stride 4, followed by batch normalization. This stage reduces the input spectrogram from $224 \times 224 \times 3$ to $56 \times 56 \times 96$ while preparing a compact representation of the frequency-time information. The mathematical definition of this stem phase is provided below.

$$T_1 = BN\left(C_{4 \times 4, \text{Stride:4}}^{96}(lm)\right)$$

The main phase of the network is composed of multiple GuitarNeXt blocks, which combine convolution operations with pooling-based attention. Inside each block, a 7×7 grouped convolution produces an initial feature map, and three parallel transformation paths are applied to capture different types of spectral patterns. These paths involve combinations of pointwise convolution, GELU activation, max pooling, and average pooling, where features are fused through element-wise multiplication to create interaction between local activations and contextual pooling cues. The outputs

of the branches are concatenated to expand the feature depth before a projection layer and residual connection refine the representation and improve gradient flow.

$$T_n = GC_{7 \times 7}^F(T_{n-1})$$

The hybrid downsampling stage reduces spatial size using both max pooling and average pooling with stride 2. These representations are concatenated and batch-normalized, allowing the model to retain complementary details that might be lost through a single downsampling operation. The mathematical definition of this phase is provided below.

$$T_d = BN\left(DC\left(MP_{2 \times 2, Stride:2}^F(T_{d-1}), DC\left(AP_{2 \times 2, Stride:2}^F(T_{d-1})\right)\right)\right)$$

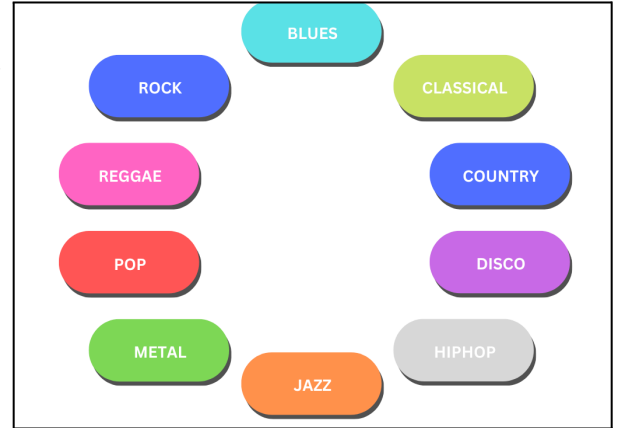
The output classification stage then applies global average pooling and global max pooling in parallel to generate a flattened representation that captures both subtle and dominant frequency patterns. This vector undergoes grouped convolution and GELU activation before being passed through a fully connected layer and softmax function to produce final predictions .

4. Implementation Details

4.1 Dataset

This project uses the 10-class GTZAN Music Genre Classification dataset shared by Andrada Olteanu on the Kaggle platform. It is one of the most popular benchmark dataset for music genre classification. It contains ten musical genres, each represented by one hundred audio excerpts that are thirty seconds long. Although commonly used, the dataset has some imperfections, such as mislabeled or corrupted files. Any corrupted files encountered during preprocessing were skipped to ensure consistent input to the model.

You can access the original dataset [here](#) and my implementation [here](#).



4.2 Spectrogram Conversion

All audio files were first resampled to a standardized sampling rate of 22,050 Hz. Short-Time Fourier Transform (STFT) analysis was applied using a 2,048-point FFT window and a hop length of 512 samples to produce 128-band mel-spectrograms. The magnitude values of the spectrogram were

converted to logarithmic scale and normalized to the range [0, 1] to improve numerical stability during training. Each spectrogram was then resized to 224×224 pixels and formatted as a three-channel RGB image.

4.3 Data Split

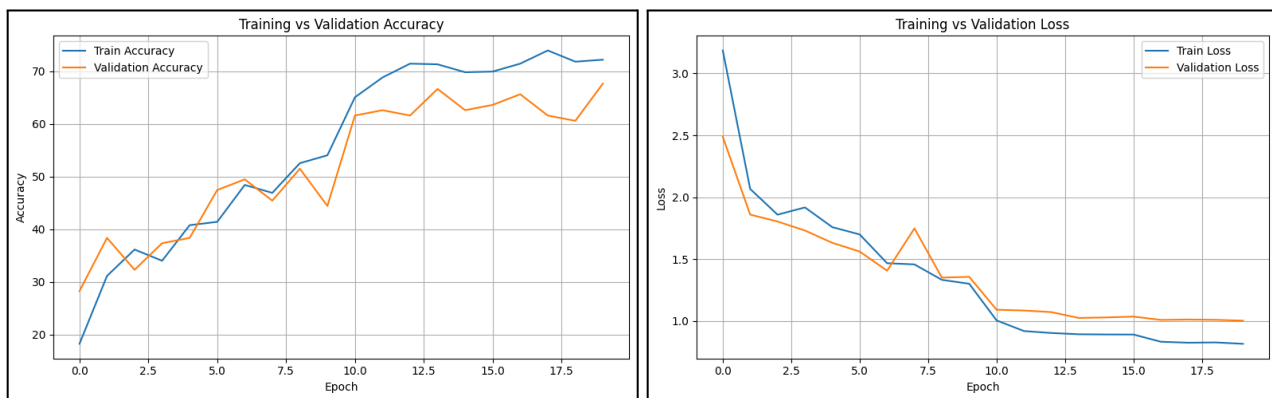
The dataset was divided into independent subsets for training, validation, and testing. Eighty percent of the samples were used to train the model, while ten percent was allocated to validation for hyperparameter tuning. The remaining ten percent was used as the final test set for evaluating the performance of the model.

4.4 Training Configuration

The GuitarNeXt model was trained using the SGD optimizer with a learning rate of 0.01 and weight decay of 0.001 to improve generalization. A MultiStepLR scheduler was applied to reduce the learning rate at epochs 10 and 20. Each training batch contained 32 spectrogram images, and the model was trained for a total of 25 epochs. To reduce overfitting and improve robustness, data augmentation techniques such as random rotation and background noise addition were applied during training.

5. Experimental Results

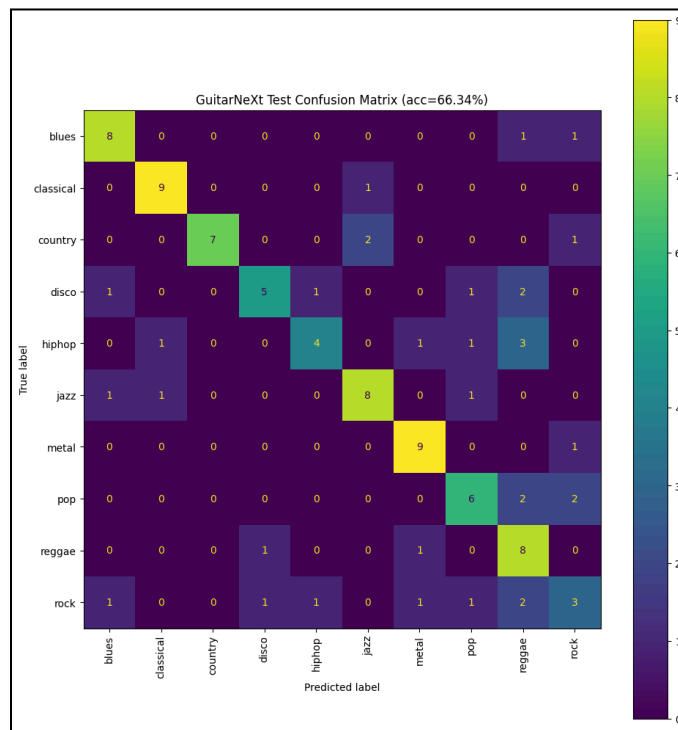
5.1 Training Results



The GuitarNeXt model was trained for 25 epochs based on validation accuracy. During training, the model achieved a maximum accuracy of approximately 74% on the training set, while validation accuracy plateaued at around 67%. The model converged at around 18 epochs and additional training didn't significantly improve validation performance.

5.2 Evaluation Metrics

On the test set, the model achieved a final accuracy of 66.34%, closely matching validation performance. To provide a more complete assessment across all genres, macro-averaged evaluation metrics were computed. The model achieved a precision of 0.6823, a recall of 0.6627, and an F1-score of 0.6581, indicating fairly balanced performance across the ten classes. Certain genres such as classical and jazz exhibited comparatively stronger recall which could be attributed to their distinctive instrumental structure. In contrast, genres like rock, disco, and hip-hop had a lot more confusion due to their similarities.



The confusion matrix revealed several consistent misclassification trends. Rock and hip-hop were often confused with Reggae. Although the model was able to classify reggae quite accurately, it also chose reggae a lot more times compared to the other genres. This may be influenced by the dataset's subpar audio quality and limited audio diversity. These patterns however suggest that musical similarities were the main source of error rather than network limitations.

5.3 Comparison to Paper Results

The original GuitarNeXt research reported classification accuracy of 96%. That model, however, was trained using a more complex three-stage training process involving pretraining on larger external datasets and synthetic spectrogram augmentation. In contrast, the model in this project was trained entirely from scratch using only the 1,000 samples in GTZAN. The absence of large-scale pretraining and limited data availability led to comparatively lower performance. Hardware constraints also

reduced the ability to expand and create a more robust implementation. Despite these differences, achieving over 66% test accuracy demonstrates that a lightweight version of GuitarNeXt remains effective for genre classification.

6. Strengths and Weaknesses

Our implementation of GuitarNeXt proved to be capable of learning meaningful musical patterns from a relatively small dataset, reaching over 66% test accuracy with balanced macro-precision, recall, and F1-score values. The model showed strong performance on genres, such as classical and jazz, and successfully avoided major overfitting despite limited data. The architecture was also computationally lightweight, enabling efficient training without requiring high-end GPU resources or extensive pretraining.

However, the system also exhibited a number of weaknesses. The limited size of the GTZAN dataset restricted the model's ability to generalize, resulting in confusion between genres with overlapping characteristics. Additionally, the lack of multi-stage pretraining, as used in the original GuitarNeXt paper, contributed to a noticeable performance gap relative to state-of-the-art results. These weaknesses suggest that larger datasets, stronger augmentation, and improved training strategies are necessary to further enhance classification accuracy.

7. Conclusion and Future Work

This project explored the implementation and evaluation of the GuitarNeXt architecture for music genre classification using the GTZAN dataset. By converting audio signals into mel-spectrogram images and training the model from scratch, the system learned to extract meaningful time-frequency patterns associated with musical styles. The model achieved a test accuracy of 66.34% with similar macro-averaged precision, recall, and F1-scores, demonstrating that GuitarNeXt can perform reasonably well even when trained under limited data conditions.

While these results fall short of the performance reported in the original GuitarNeXt paper, the disparity is largely attributable to differences in training scale. The original model benefits from a multi-stage training procedure and large external pretraining datasets, while this implementation relies solely on GTZAN.

Future work may focus on improving generalization and closing the performance gap with published results. Incorporating a three-stage training pipeline including large-scale pretraining could significantly boost accuracy. Stronger data augmentation, such as SpecAugment, pitch shifting, and tempo variation, may help the model better handle audio variability. Expanding the dataset to include more recordings per genre or integrating modern transformer-based components may also further

enhance performance. Overall, this project establishes a solid foundation for applying GuitarNeXt to spectrogram-based classification and highlights promising directions for continued research.

8. References

- [1] P. J. Du, Z. Zhou, and F. Q. Li, “GuitarNeXt: An advanced convolutional neural network architecture for music genre classification,” *Traitement du Signal*, vol. 42, no. 4, pp. 2205–2214, 2025, doi: 10.18280/ts.420431.
- [2] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002, doi: 10.1109/TSA.2002.800560.
- [3] O. Andrada, *GTZAN Music Genre Dataset*. Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>
- [4] B. McFee *et al.*, “librosa/librosa: 0.11.0,” *Zenodo*, 2025, doi: 10.5281/zenodo.15006942.
- [5] N. Pourmoazemi and S. Maleki, “A music recommender system based on compact convolutional transformers,” *Expert Systems with Applications*, vol. 255, 124473, 2024, doi: 10.1016/j.eswa.2024.124473.
- [6] H. Wang, S. SalmiJamali, Z. Chen, Q. Shang, and L. Ran, “An intelligent music genre analysis using feature extraction and classification using deep learning techniques,” *Computers & Electrical Engineering*, vol. 100, 107978, 2022.
- [7] Y. Zhang and T. Li, “Music genre classification with parallel convolutional neural networks and capuchin search algorithm,” *Scientific Reports*, vol. 15, 9580, 2025, doi: 10.1038/s41598-025-90619-7.